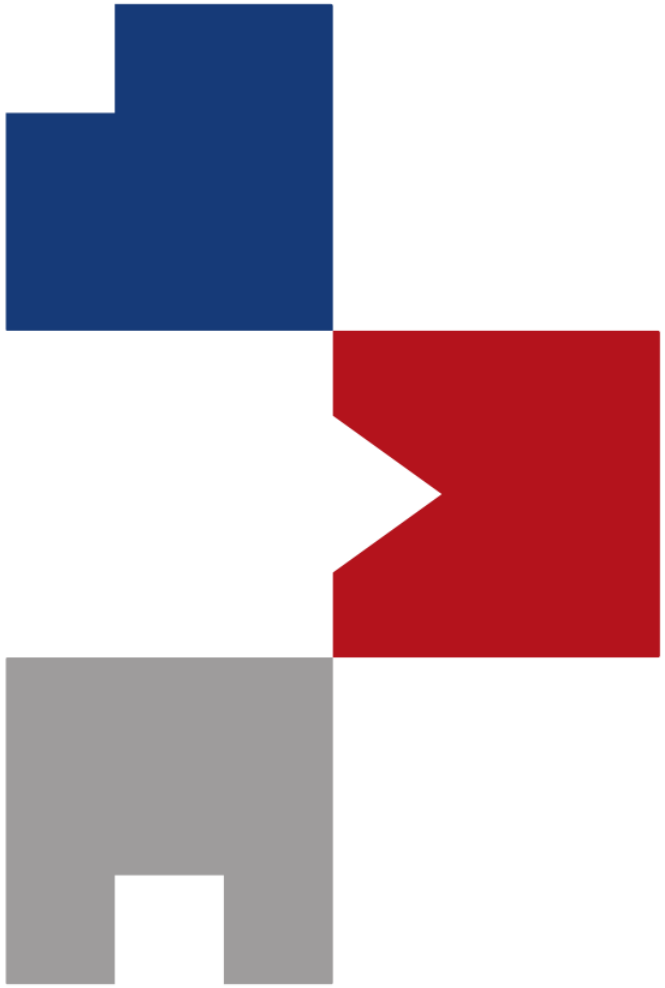
Note) Slides and example codes are available:
https://github.com/mint-lab/dl_tutorial



ML Lab #1:
Breast Cancer Classification

Sunglok Choi, Assistant Professor, Ph.D.
Dept. of Computer Science and Engineering, SEOULTECH
sunglok@seoultech.ac.kr | https://mint-lab.github.io/

Overview

- **Prerequisite**
  - Anaconda (Individual Edition)



Pinkwashing

- **Practice) Breast Cancer Classification**
  - The given data
  - Expected results
  - Practice with the skeleton code
    - Step #1) Load the dataset
    - Step #2) Find any better classifier
    - Step #3) Visualize the confusion matrix

- **Assignment**
  - Mission: Complete the given skeleton code

Practice) Breast Cancer Classification

- The given data: [Breast Cancer Wisconsin (Diagnostic) Data Set]
  - Classes (#: **2**): *Malignant* (M; 악성종양 in Korean), *Benign* (B; 양성종양)
  - Attributes: **30** real numbers (except ID and target class)
    - Radius
    - Texture
    - Perimeter
    - Area
    - …
  - The number of data: **569** (M: 212, B: 357)
  - Note) Load the dataset using scikit-learn [API]

```
from sklearn import datasets
wdbc = datasets.load_breast_cancer()
```



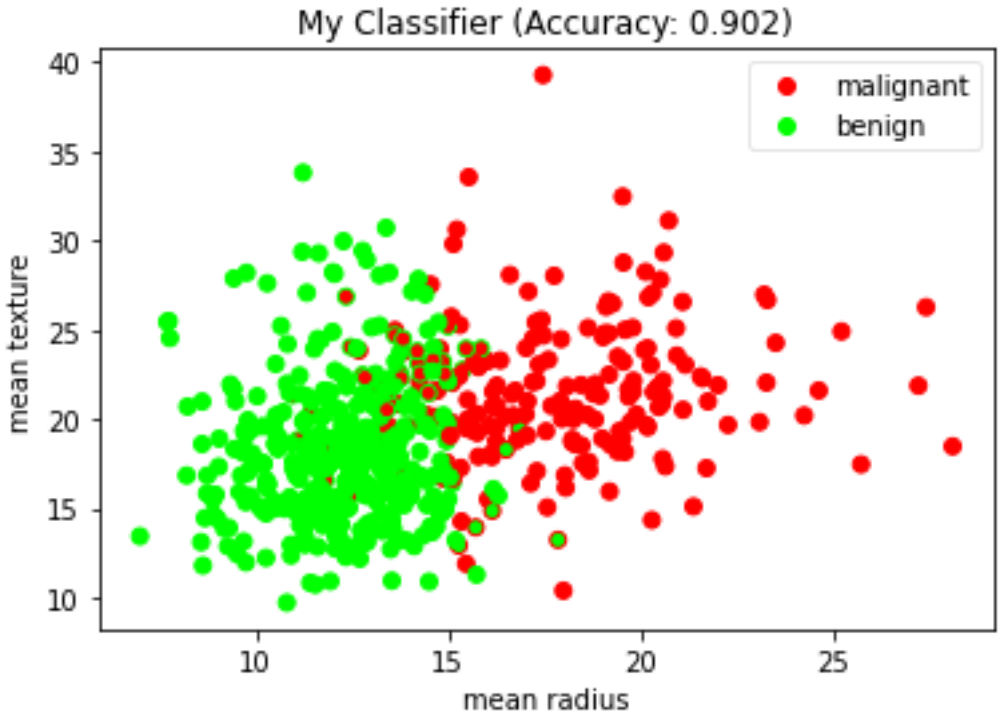
Practice) Breast Cancer Classification

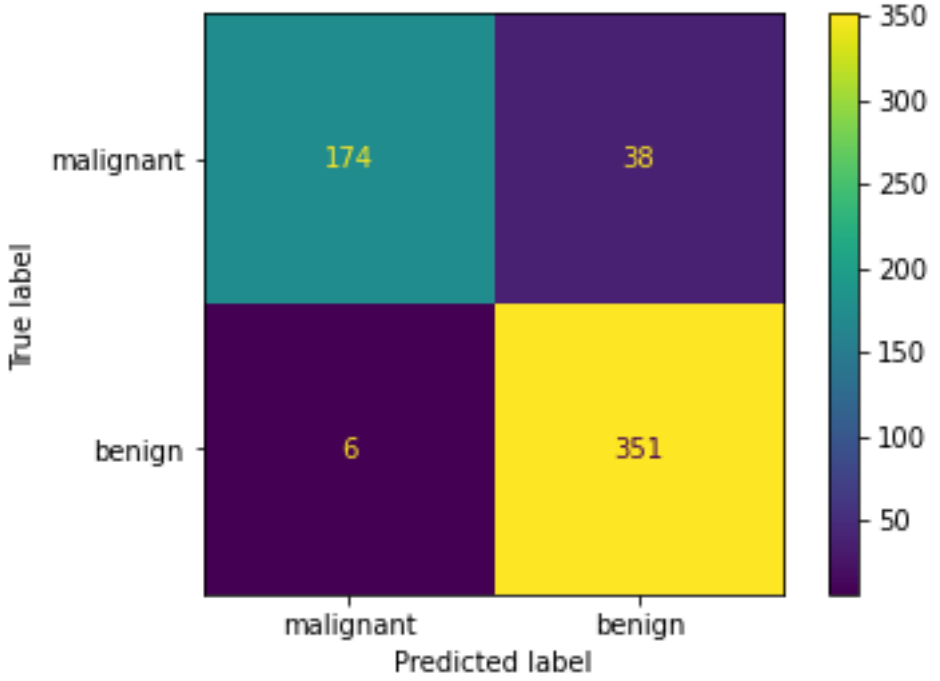
- The given data (file: `data/wdbc.data`)
  - File format: CSV (comma-separated values)
    - `ID,target class (M or F),radius,texture,perimeter,area, ...`
  - Example

```
842302,M,17.99,10.38,122.8,1001,0.1184,0.2776,0.3001,0.1471,0.2419,0.07871,1.095,0.9053,8.589,15
3.4,0.006399,0.04904,0.05373,0.01587,0.03003,0.006193,25.38,17.33,184.6,2019,0.1622,0.6656,0.711
9,0.2654,0.4601,0.1189
...
```

Practice) Breast Cancer Classification

- Expected results
  - The default classifier: SVM (`svm.SVC`)





Practice) Breast Cancer Classification

- The given skeleton code (`wdbc_classification_skeleton.py`)
  - Step #1) Load the dataset

```
def load_wdbc_data(filename):
    class WDBCData:
        data         = [] # Shape: (569, 30)
        target       = [] # Shape: (569, )
        target_names = ['malignant', 'benign']
        ...
    wdbc = WDBCData()
    with open(filename) as f:
        for line in f.readlines():
            items = line.split(',')
            wdbc.target.append(items[1])        # TODO #1) Add the true label (0 for M / 1 for others)
            wdbc.data.append(items[2:])         # TODO #1) Add 30 attributes (as floating-point numbers)
        wdbc.data = np.array(wdbc.data)
    return wdbc

if __name__ == '__main__':
    # Load a dataset
    wdbc = load_wdbc_data('data/wdbc.data')     # TODO #1) Implement 'load_wdbc_data()'
```

Practice) Breast Cancer Classification

- The given skeleton code (`wdbc_classification_skeleton.py`)
  - Step #2) Find any better classifier
  - Step #3) Visualize the confusion matrix

```
if __name__ == '__main__':
    # Load a dataset
    # wdbc = datasets.load_breast_cancer()
    wdbc = load_wdbc_data('data/wdbc.data')     # TODO #1) Implement 'load_wdbc_data()'

    # Train a model
    model = svm.SVC()                           # TODO #2) Find a better classifier (SVC accuracy: 0.902)
    model.fit(wdbc.data, wdbc.target)

    # Test the model
    predict = model.predict(wdbc.data)
    accuracy = metrics.balanced_accuracy_score(wdbc.target, predict)

    # Mission #3) Visualize the confusion matrix

    # Visualize testing results
    ...
```

Assignment

- Mission
  - Complete the following three missions using the given skeleton code (`wdbc_classification_skeleton.py`)
  - Submit your code (`wdbc_classification.py`) and its two result images (`wdbc_classification_scatter.png`, `wdbc_classification_matrix.png`)

- Condition
  - Please follow the above filename convention.
  - You can start from scratch (without using the given skeleton code).
    - However, you should use the given data.
  - You can freely change the given skeleton code if necessary.

- Submission
  - Where: e-Class > Assignments
  - Score: Max 10 points